Updated on: May 2, 2024 11 min read

The ultimate guide to SBOMs

Learn what a software bill of materials is and why it has become an integral part of modern software development.

Sandra Gittlen

security DevSecOps performance open source public sector

In today's rapidly evolving digital landscape, the emphasis on application security within the software supply chain has never been more critical. The integration of upstream dependencies into software requires transparency and security measures that can be complex to implement and manage. This is where a software bill of materials (SBOM) becomes indispensable.

Serving as a comprehensive list of ingredients that make up software components, an SBOM illuminates the intricate web of libraries, tools, and processes used across the development lifecycle. Coupled with vulnerability management tools, an SBOM not only reveals potential vulnerabilities in software products but also paves the way for strategic risk mitigation. Our guide dives deep into SBOMs, their pivotal role in a multifaceted DevSecOps strategy, and strategies for improving your application's SBOM health — all aimed at fortifying your organization's cybersecurity posture in a landscape full of emerging threats.

You'll learn:

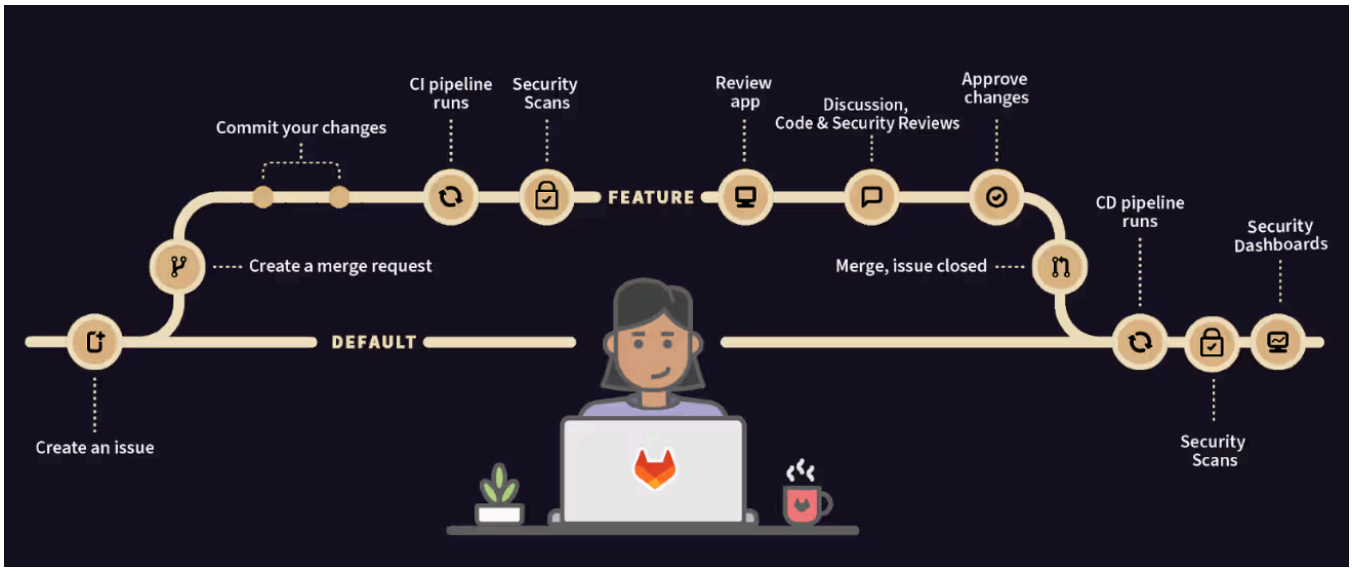
- What is an SBOM?
- Why SBOMs are important
- Types of SBOM data exchange standards
- Benefits of pairing SBOMs and software vulnerability management
- GitLab and dynamic SBOMs
  - Scale SBOM generation and management
  - Ingest and merge SBOMs
  - Accelerate mitigation for better SBOM health
  - Continuous SBOM analysis
  - Building trust in SBOMs
- The future of GitLab SBOM functionality
- Get started with SBOMs
- SBOM FAQ

What is an SBOM?

An SBOM is a nested inventory or list of ingredients that make up software components. In addition to the components themselves, SBOMs include critical information about the libraries, tools, and processes used to develop, build, and deploy a software artifact.

The SBOM concept has existed for more than a decade. However, as part of an effort to implement the National Cyber Strategy that the White House released in 2023, CISA's Secure by Design framework is helping guide software manufacturers to adopt secure-by-design principles and integrate cybersecurity into their products. The U.S. government issued best practices that are driving application developers selling to the public sector to include SBOMs with their software packages. The private sector is not far behind, sending SBOMs on the path to ubiquity.

Although SBOMs are often created with stand-alone software, platform companies like GitLab are integrating SBOM generation early and deep in the DevSecOps workflow.



Why SBOMs are important

Modern software development is laser-focused on delivering applications at a faster pace and in a more efficient manner. This can lead to developers incorporating code from open source repositories or proprietary packages into their applications. According to Synopsys's 2024 Open Source Security and Risk Analysis report, which consolidated findings from more than 1,000 commercial codebases across 17 industries in 2023, 96% of the total codebases contained open source and 84% of codebases assessed for risk contained vulnerabilities.

Pulling in code from unknown repositories increases the potential for vulnerabilities that can be exploited by hackers. In fact, the 2020 SolarWinds attack was sparked by the activation of a malicious injection of code in a package used by SolarWinds' Orion product.

Customers across the software supply chain were significantly impacted. Other attacks, including the log4j vulnerability that impacted a number of commercial software vendors, cemented the need for a deep dive into application dependencies, including containers and infrastructure, to be able to assess risk throughout the software supply chain.

There is also a cost component to finding and remediating a software security vulnerability that levels up the need for SBOMs, as well as damage to a company's reputation that a software supply chain attack can incur. SBOMs give you insight into your dependencies and can be used to look for vulnerabilities, and licenses that don't comply with internal policies.

Types of SBOM data exchange standards

SBOMs work best when their generation and interpretation of information such as name, version, packager, and more are able to be automated. This happens best if all parties use a standard data exchange format.

There are two main types of SBOM data exchange standards in use today:

- OWASP CycloneDX
- SPDX

GitLab uses CycloneDX for its SBOM generation because the standard is prescriptive and user-friendly, can simplify complex relationships, and is extensible to support specialized and future use cases. In addition, cyclonedx-cli and cdx2spdx are open source tools that can be used to convert CycloneDX files to SPDX if necessary.

Benefits of pairing SBOMs and software vulnerability management

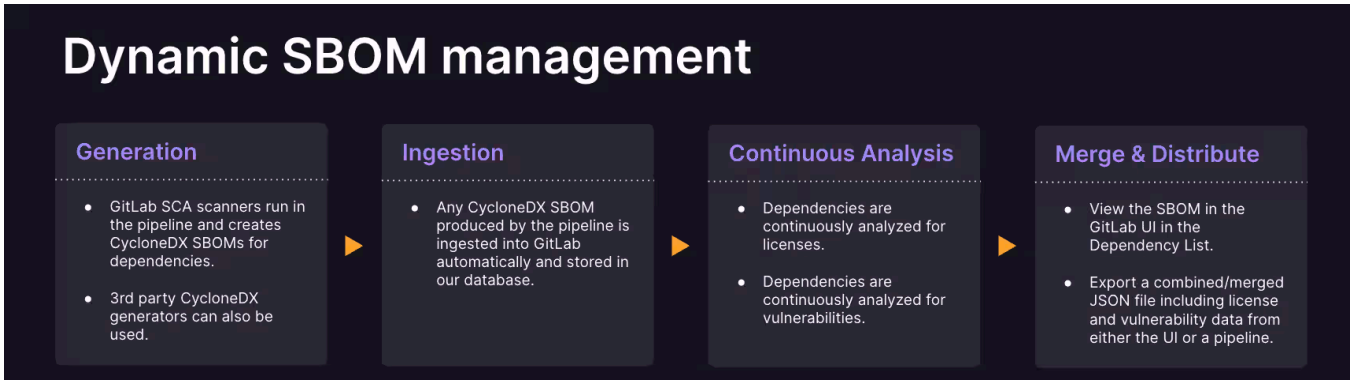
SBOMs are highly beneficial for DevSecOps teams and software consumers for several reasons:

- They enable a standard approach to understanding what additional software components are in an application and where they are declared.
- They provide ongoing visibility into the history of an application's creation, including details about third-party code origins and host repositories.
- They provide a deep level of security transparency into both first-party developed code and adopted open source software.

- The details that SBOMs offer enable a DevOps team to identify vulnerabilities, assess the potential risks, and then mitigate them.
- SBOMs can deliver the transparency that application purchasers now demand.

GitLab and dynamic SBOMs

For SBOMs to be fully impactful, organizations must be able to automatically generate them, connect them with application security scanning tools, integrate the vulnerabilities and licenses into a dashboard for easy comprehension and actionability, and update them continuously. GitLab supports all of these goals.



Scale SBOM generation and management

To comply with internal policies and regulations, it is key to have accurate and comprehensive SBOMs that cover open source, third-party, and proprietary software. To effectively manage SBOMs for each component and product version, a streamlined process is required for creating, merging, validating and approving SBOMs. GitLab's Dependency List feature aggregates known vulnerability and license data into a single view within the GitLab user interface. Dependency graph information is also generated as part of the dependency scanning report. This empowers users to gain comprehensive insights into dependencies and risk within their projects or across groups of projects. Additionally, a JSON CycloneDX formatted artifact can be produced in the CI pipeline. This API introduces a more nuanced and customizable approach to SBOM generation. SBOMs are exportable from the UI, a specific pipeline or project, or via the GitLab API.

Ingest and merge SBOMs

GitLab can ingest third-party SBOMs, providing a deep level of security transparency into both third-party developed code and adopted open source software. With GitLab, you can use a CI/CD job to seamlessly merge multiple CycloneDX SBOMs into a single SBOM.

Using implementation-specific details in the CycloneDX metadata of each SBOM, such as the location of build and lock files, duplicate information is removed from the resulting merged file. This data is also augmented automatically with license and vulnerability information for the components inside the SBOM.

Accelerate mitigation for better SBOM health

Building high-quality products faster requires actionable security findings so developers can address the most critical weaknesses. GitLab helps secure your supply chain by scanning for vulnerabilities in source code, containers, dependencies, and running applications. GitLab offers full security scanner coverage from Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), container scanning, and software composition analysis (SCA) features to help you achieve full coverage against emerging threat vectors. To help developers and security engineers better understand and remediate vulnerabilities more efficiently, GitLab Duo Vulnerability Explanation, an AI-powered feature, provides an explanation about a specific vulnerability, how it can be exploited, and, most importantly, a recommendation on how to fix the vulnerability. When combined with GitLab Duo Vulnerability Resolution, DevSecOps teams can intelligently identify, analyze, and fix vulnerabilities in just a matter of clicks.

The platform also supports creation of new policies (and compliance enforcement) based on newly detected vulnerabilities.

Continuous SBOM analysis

GitLab Continuous Vulnerability Scanning triggers a scan on all projects where either container scanning, dependency scanning, or both, are enabled independent of a pipeline. When new Common Vulnerabilities and Exposures (CVEs) are reported to the National Vulnerability Database (NVD), users don't need to re-run their pipelines to get the latest feeds. GitLab's Vulnerability Research Team adds them to GitLab's Advisory Database and those advisories are automatically reported up to GitLab as vulnerabilities. This makes GitLab's SBOM truly dynamic in nature.

Building trust in SBOMs

Organizations that require compliance functionality can use GitLab to generate attestation for all build artifacts produced by the GitLab Runner. The process is secure because it is produced by the GitLab Runner itself with no handoff of data to an external service.

The future of GitLab SBOM functionality

Software supply chain security continues to be a critical topic in the cybersecurity and software industry due to frequent attacks on large software vendors and the focused efforts of attackers on the open source software ecosystem. And although the SBOM industry is evolving quickly, there are still concerns around how SBOMs are generated, the frequency of that generation, where they are stored, how to combine multiple SBOMs for complex applications, how to analyze them, and how to leverage them for application health.

GitLab has made SBOMs an integral part of its software supply chain direction and continues to improve upon its SBOM capabilities within the DevSecOps platform, including planning new features and functionality. Recent enhancements to SBOM capabilities include the automation of attestation, digital signing for build artifacts, and support for externally generated SBOMs.

GitLab has also established a robust SBOM Maturity Model within the platform that involves steps such as automatic SBOM generation, sourcing SBOMs from the development environment, analyzing SBOMs for artifacts, and advocating for the digital signing of SBOMs. GitLab also plans to add automatic digital signing of build artifacts in future releases.

Get started with SBOMs

The demand for SBOMs is already high. Government agencies increasingly recommend or require SBOM creation for software vendors, federal software developers, and even open source communities.

> To get ahead of this requirement, check out the SBOM capabilities for GitLab Ultimate in GitLab's DevSecOps platform.

SBOM FAQ

What is an SBOM?

An SBOM is a detailed inventory that lists all components, libraries, and tools used in

Why are SBOMs important?

SBOMs are crucial for several reasons. They provide:

- Insight into dependencies: Understanding what makes up your software helps identify and mitigate risks associated with third-party components.
- Enhanced security: With detailed visibility into application components, organizations can pinpoint vulnerabilities quickly and take steps to address them.
- Regulatory compliance: Increasingly, regulations and best practices recommend or require an SBOM for software packages, particularly for those in the public sector.
- Streamlined development: Developers can lean on an SBOM for insights into used libraries and components, saving time and reducing errors in the development cycle.

What standards are used for SBOM data exchange?

There are two predominant standards:

- CycloneDX: Known for its user-friendly approach, CycloneDX simplifies complex relationships between software components and supports specialized use cases.
- SPDX: Another widely used framework for SBOM data exchange, providing detailed information about components within the software environment.

GitLab specifically employs CycloneDX for its SBOM generation because of its prescriptive nature and extensibility to future needs.

What is GitLab's approach to SBOMs?

GitLab emphasizes the creation of dynamic SBOMs that can be:

- Automatically generated: Ensuring up-to-date information on software composition.
- Integrated with tools: Connecting to vulnerability scanning tools for thorough risk assessment.
- Easily managed: Supporting ingestion and merging of SBOMs for comprehensive analysis.
- Continuously analyzed: Offering ongoing scanning of projects to detect new vulnerabilities as they emerge.

How can I start implementing SBOMs in my organization?

For organizations ready to adopt SBOMs, GitLab's Ultimate package provides a robust platform for generating and managing SBOMs within a DevSecOps workflow. By leveraging GitLab's tools, teams can ensure compliance, enhance security, and optimize development practices.

The increasing demand for SBOMs reflects the growing emphasis on software security and supply chain integrity. By integrating SBOM capabilities, organizations can better protect themselves against vulnerabilities and comply with emerging regulations.

Try GitLab Ultimate free today.

Disclaimer This blog contains information related to upcoming products, features, and functionality. It is important to note that the information in this blog post is for informational purposes only. Please do not rely on this information for purchasing or planning purposes. As with all projects, the items mentioned in this blog and linked pages are subject to change or delay. The development, release, and timing of any products, features, or functionality remain at the sole discretion of GitLab.

More to explore

View all blog posts >

Security

Migrate from pipeline variables to pipeline inputs for better security

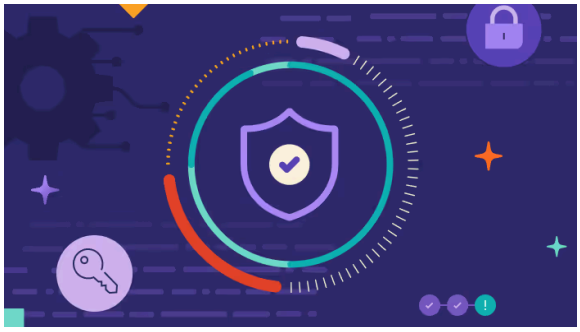
Read the blog >

Security

Delivering faster and smarter scans with Advanced SAST

Read the blog >

Security



GitLab Patch Release: 18.4.2, 18.3.4, 18.2.8

Read the blog >

We want to hear from you

Enjoyed reading this blog post or have questions or feedback? Share your thoughts by creating a new topic in the GitLab community forum.

Share your feedback >